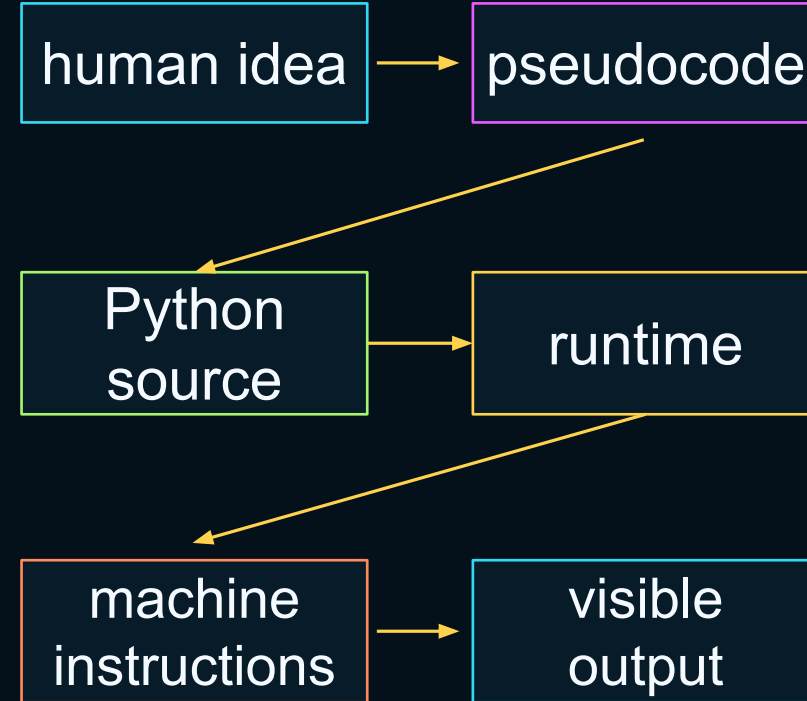


FROM HUMAN INSTRUCTIONS TO PYTHON CODE

Turning “what I want” into precise steps a computer can run



```
print("Hello, world!")
```

30 min instruction • 30 min guided Python execution

Bridge from the morning: layers matter

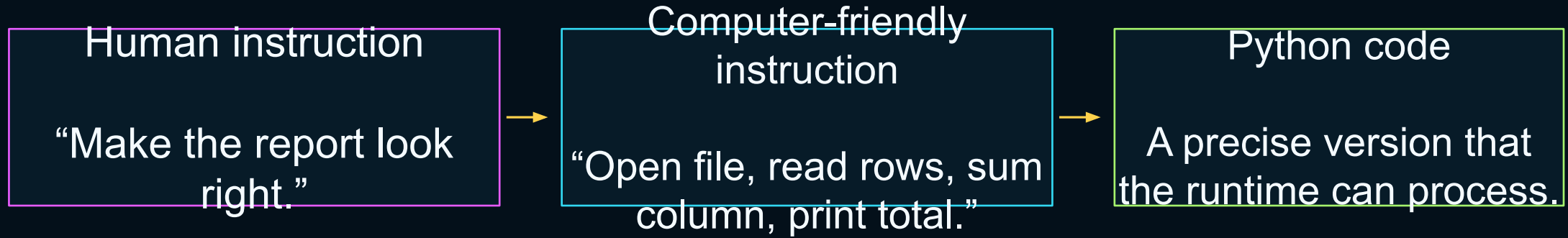
Python is several layers above the processor, memory, storage, and networking details.



The programmer works mostly at the top. Python and the operating system help with the layers below.

A program is a precise set of instructions

Computers are powerful, fast, and extremely literal.



Programming is the craft of removing ambiguity.

Algorithms: the plan before the code

An algorithm is a step-by-step method for solving a problem.

REAL LIFE

Make coffee

1. Add water
2. Add grounds
3. Press start
4. Pour cup

DATA TASK

Calculate total

1. Start at zero
2. Read each price
3. Add it to total
4. Show result

PYTHON GOAL

Translate the plan
into valid source
code the interpreter
can run.

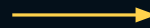
Good code usually starts as a clear plan.

Pseudocode: thinking out loud before syntax

Pseudocode is not Python. It is a readable sketch of the logic.

```
Start with total = 0
For each price in the
order:
    add price to total
Display total
```

pseudocode



```
total = 0
for price in prices:
    total = total +
price
print(total)
```

Python source code

Pseudocode lets you solve the thinking problem before the syntax problem.

Source code and programming languages

A programming language gives humans a structured way to write executable ideas.

SOURCE CODE

Text written by a
programmer

SYNTAX

Rules for what valid
code looks like

MEANING

What the instructions
do when run

OUTPUT

Visible result or
changed state

```
print("The answer is", 2 + 3)
```

The code is text, but it follows rules so Python can interpret it.

Why Python is called “high-level”

Python lets programmers express ideas without managing every machine detail.

```
print("Hello,  
world!")
```

high-level: human readable

```
10110000 01100001  
01001000 10001001
```

low-level: close to machine representation

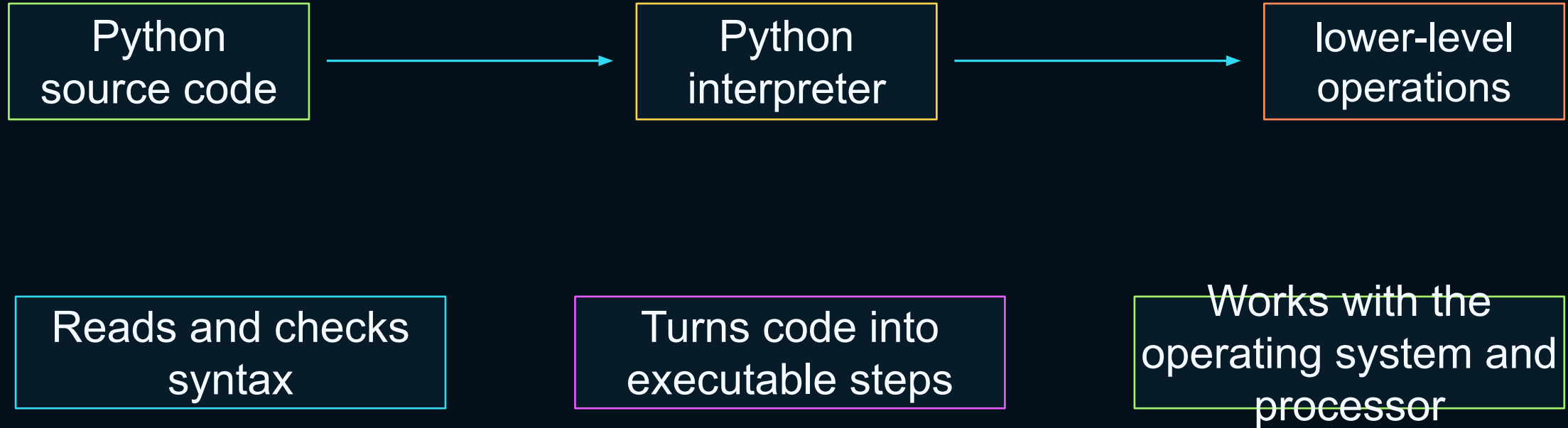
Less hardware detail

More readable
programs

Faster development

The Python interpreter: the code runner

The interpreter reads Python source code and performs the work needed to run it.



Python bytecode: a lower-level middle step

Python usually converts source code into bytecode before execution.



Bytecode is not the same as
processor machine code.

It is a lower-level set of instructions
for the Python runtime.

Practical takeaway:

You write Python. Python handles
the boring translation layers.

Three common ways to run Python

The same language can be used in different working styles.

INTERACTIVE

Run one command at a time.
Great for trying things quickly.

```
>>> print("hi")
```

SCRIPT

Save code in a .py file.
Run the file as a repeatable program.

```
hello.py
```

NOTEBOOK

Run code in cells with notes and outputs.
Great for learning and analysis.

cell → output

Today: comfort first. Run code, observe output, fix simple mistakes.

Statements, expressions, and comments

These three building blocks appear immediately in beginner Python.

```
# This is a comment  
print("Python is running.")  
2 + 3  
print("The answer is", 2 + 3)
```

COMMENT

Ignored by Python; helps humans read code.

STATEMENT

An instruction that Python executes.

EXPRESSION

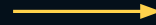
A piece of code that produces a value.

Beginner shortcut: “Does it do something?” statement. “Does it produce a value?” expression.

Execution order: Python runs top to bottom

Unless told otherwise, Python executes one statement at a time in order.

```
print("first")  
print("second")  
print("third")
```



```
first  
second  
third
```

Reading code in order is one of the first debugging superpowers.

Demo: anatomy of a first Python line

One line contains a function name, data, and syntax.

```
print("Hello, world!")
```

function name

opening parenthesis

quote begins text

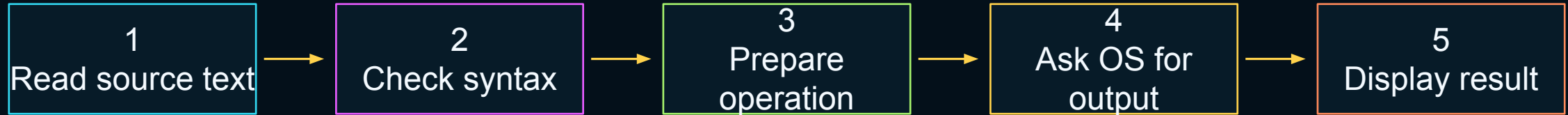
data: text value

syntax closes the call

Python is picky about punctuation because the punctuation gives the code structure.

What happens after you press Run?

Even a simple line goes through multiple layers.



```
print("Hello, world!")
```

Hello, world!

Class discussion: name the pieces

Use the vocabulary before running more code.

```
print("The answer is", 2 + 3)
```

function name?

text data?

numeric expression?

syntax?

Expected output: The answer is 5

Guided exercise round 1: predict, then run

Before pressing Run, write down what you expect to appear.

```
print("Python is running.")  
print(2 + 3)  
print("The answer is", 2 + 3)
```

Task A: predict the output

Task B: run the code

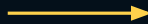
Task C: compare prediction to reality

Rule for today: prediction first, execution second.

Round 1 answer reveal

The output appears in the order Python executes the lines.

```
print("Python is running.")  
print(2 + 3)  
print("The answer is", 2 + 3)
```



```
Python is running.  
5  
The answer is 5
```

Notice: Python evaluates $2 + 3$ before printing the final line.

Guided exercise round 2: comments, numbers, and text

Run each line and notice what changes the output.

```
# A comment for humans  
print("Hello")  
print(10)  
print("10")  
print(10 + 5)  
print("10 + 5")
```

Which line is ignored?

Which lines print numbers?

Which lines print text exactly?

Text in quotes is data. Arithmetic outside quotes is calculated.

Guided exercise round 3: intentionally break it

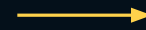
Errors are feedback from the interpreter, not a personal failure.

```
print("This line is broken)
```



Missing closing quote
Python cannot tell where the text ends.

```
print("This line is fixed")
```



Closing quote restored
The syntax now has structure.

EXERCIS

Mini-lab checklist

Students complete these in their Python environment.

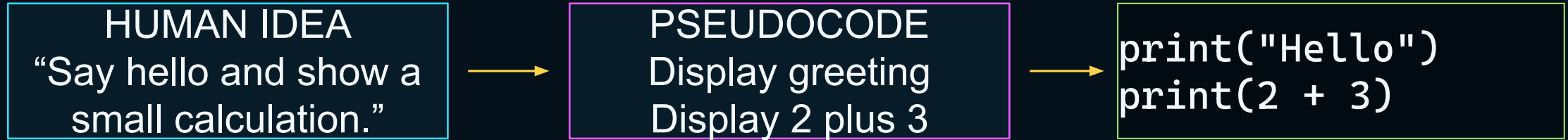
- 1 Open** the Python environment
- 2 Run** at least three `print()` statements
- 3 Print** both numbers and text
- 4 Add** one comment above a line of code
- 5 Break** one line with a small syntax error
- 6 Fix** the error and run again

Instructor goal

Keep students moving. The win is confidence with
run → observe → adjust.

Same human idea, different precision levels

This is the pattern students will use all course long.



```
Hello
5
```

visible output

Wrap-up discussion

Close with the mental model students need for the next block.

What is the difference between an algorithm and Python source code?

Why is Python easier for humans than machine code?

What does the interpreter do for the programmer?

Why should we predict output before running code?

Why are syntax errors useful instead of just annoying?

Next up: values, variables, and the first useful building blocks of Python programs.